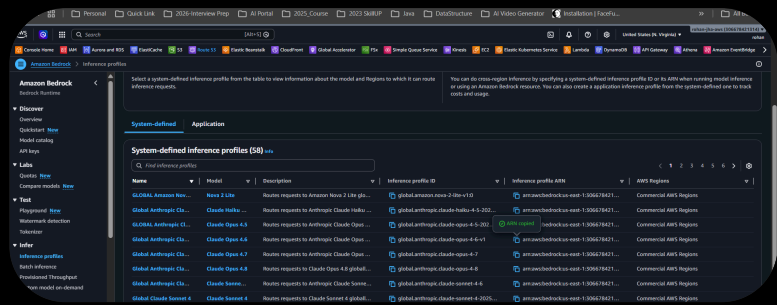




```
aws bedrock using spring AI >> getting error
> java.lang.IllegalArgumentException: Parameter 'modelId'
must not be null >> spring:
application:
  name: aws-bedrock

ai:
  model:
    chat: bedrock-converse
  tool:
    enabled: false

Show more
```

The error:

```
</> Java
```

```
java.lang.IllegalArgumentException: Parameter 'modelId' must not be null
```

usually means Spring AI is **not loading the Bedrock model configuration**, so when it creates the `BedrockConverseChatModel`, the `modelId` property is `null`.

Looking at your YAML, I suspect you're using the wrong property hierarchy.

1. Check Spring AI Version

Which version are you using?

For Spring AI 1.0.x / 1.1.x, the configuration is typically:

```
</> YAML
```

```
spring:
  ai:
    bedrock:
      aws:
        region: us-east-1
```

```
    access-key: xxx
    secret-key: xxx

  converse:
    chat:
      options:
        model: arn:aws:bedrock:us-east-1:306678421314:inference-profile/global.anthro
```

Notice:

YAML



```
options:
  model:
```

not

YAML



```
chat:
  model:
```

2. Verify Dependency

If using Bedrock Converse API, dependency should be something like:

XML



```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-model-bedrock-converse</artifactId>
</dependency>
```

NOT:

XML



```
spring-ai-starter-model-bedrock-anthropic
```

because Converse and Anthropic starters are different.

3. Debug Loaded Properties

Add:

</> Java



```
@Component
@RequiredArgsConstructor
public class TestRunner implements CommandLineRunner {

    @Value("${spring.ai.bedrock.converse.chat.options.model:NULL}")
    private String model;

    @Override
    public void run(String... args) {
        System.out.println("MODEL = " + model);
    }
}
```

If you see:

MODEL = NULL



then Spring Boot isn't binding the property.

4. ARN vs Inference Profile ID

From your screenshot:

Inference Profile ID:

global.anthropic.claude-opus-4-6-v1



Try using the ID instead of the ARN:

</> YAML



```
spring:
  ai:
    bedrock:
      converse:
        chat:
          options:
            model: global.anthropic.claude-opus-4-6-v1
```

Many Bedrock SDK examples expect the **Inference Profile ID** as the model identifier.

5. Verify Spring AI Sees the Bean

Add:

</> Java



```
@Autowired
ChatModel chatModel;

@PostConstruct
public void init() {
    System.out.println(chatModel.getClass());
}
```

Expected:

```
class org.springframework.ai.bedrock.converse.BedrockConverseChatModel
```



If not, Spring AI isn't creating the Bedrock Converse model.

6. Most Common Cause

If you're on Spring AI 1.0+ and copied the YAML above, this is likely wrong:

</> YAML



```
spring:
  ai:
    bedrock:
      converse:
        chat:
          model: arn:....
```

and should be:

</> YAML



```
spring:
  ai:
    bedrock:
      converse:
        chat:
          options:
            model: global.anthropic.claude-opus-4-6-v1
```

Can you share:

1. `pom.xml` dependencies related to Spring AI
2. Spring Boot version

3. Spring AI version

Then I can tell you the exact YAML structure required for that version.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.3</version>
```

Show more ▾

Your dependency setup looks correct for **Spring Boot 3.5.3 + Spring AI 1.1.0 + Bedrock Converse**.

The problem is most likely your **application.yml** configuration.

For Spring AI 1.1.0, try this exact configuration:

YAML

```
spring:
  application:
    name: aws-bedrock

  ai:
    model:
      chat: bedrock-converse

  bedrock:
    aws:
      region: us-east-1
      access-key: ${AWS_ACCESS_KEY}
      secret-key: ${AWS_SECRET_KEY}

  converse:
    chat:
      options:
        model: global.anthropic.claude-opus-4-6-v1
```

Notice:

</> YAML



```
options:
  model:
```

NOT

</> YAML



```
chat:
  model:
```

Also verify what Spring is loading

Add:

</> Java



```
@Component
public class StartupCheck implements CommandLineRunner {

    @Value("${spring.ai.bedrock.converse.chat.options.model:NOT_FOUND}")
    private String model;

    @Override
    public void run(String... args) {
        System.out.println("MODEL => " + model);
    }
}
```

If startup prints:

```
MODEL => NOT_FOUND
```



then Spring isn't reading your YAML.

Another important thing

You copied the **Inference Profile ARN** from Bedrock:

```
arn:aws:bedrock:us-east-1:306678421314:inference-profile/global.anthropic.claude-op
```



Spring AI expects the **model id**.

Use:

</> YAML



```
model: global.anthropic.claude-opus-4-6-v1
```

instead of the ARN.

Check your controller

How are you injecting the model?

Correct:

</> Java



```
@RestController
public class ChatController {

    private final ChatClient chatClient;

    public ChatController(ChatClient.Builder builder) {
        this.chatClient = builder.build();
    }

    @GetMapping("/chat")
    public String chat(@RequestParam String prompt) {
        return chatClient.prompt(prompt)
            .call()
            .content();
    }
}
```

Enable full auto-configuration report

Add:

</> YAML



```
debug: true
```

and restart.

Look for:



in startup logs.

Can you also paste the **full stacktrace starting from "Caused by:"** and your **ChatController / ChatClient bean code**? The exact line throwing `modelId must not be null` will reveal whether it's:

1. YAML binding issue
2. Wrong property name
3. ARN vs model ID issue
4. ChatClient bean creation issue

My bet is **#2 + #3** based on the YAML you showed.

ok abhi work kar raha hai, but ye bedrock ke sath spring ai kaise kaam keya abhi? flow kya hai?

Abhi jo tumne setup kiya hai, usme **Spring AI sirf ek abstraction layer hai**. Actual LLM call **Amazon Bedrock** ko ja rahi hai, aur Bedrock us request ko **Claude Opus 4.6** tak route kar raha hai.

Flow ko Java developer perspective se samjho:

1. Tumhara Controller

</> Java



```
@GetMapping("/chat")
public String chat(String prompt) {
    return chatClient.prompt(prompt)
        .call()
        .content();
}
```

Yahan tum Spring AI ke `ChatClient` ko call kar rahe ho.

2. ChatClient

Spring AI internally:

Java



```
ChatClient
    -> ChatModel
        -> BedrockConverseChatModel
```

create karta hai.

Tumne yaml me:

YAML



```
spring:
  ai:
    model:
      chat: bedrock-converse
```

diya hai.

Isliye Spring Boot startup ke time:

Java



```
BedrockConverseChatModel
```

bean create karta hai.

3. BedrockConverseChatModel

Ye class AWS SDK use karti hai.

Internally kuch aisa hota hai:

Java



```
ConverseRequest request =
    ConverseRequest.builder()
        .modelId("global.anthropic.claude-opus-4-6-v1")
        .messages(...)
        .build();
```

Aur fir:

Java



```
bedrockRuntimeClient.converse(request);
```

call hota hai.

4. AWS SDK

Spring AI khud Bedrock ko HTTP call nahi karta.

Ye AWS SDK use karta hai:

</> Java



```
software.amazon.awssdk.services.bedrockruntime
```

SDK credentials uthata hai:

</> YAML



```
access-key
secret-key
region
```

Aur signed request generate karta hai.

5. Bedrock

AWS Bedrock receive karta hai:

</> JSON



```
{
  "modelId": "global.anthropic.claude-opus-4-6-v1",
  "messages": [
    {
      "role": "user",
      "content": "Hello"
    }
  ]
}
```

6. Inference Profile

Tumne direct Claude model nahi diya.

Tumne diya:

```
global.anthropic.claude-opus-4-6-v1
```



Ye actually:

Inference Profile

hai.

Bedrock internally decide karta hai:

```
US-East-1
US-West-2
EU-West-1
...
```



kis region me request bhejni hai.

Benefits:

- Cross-region failover
- Better latency
- Higher throughput

7. Anthropic Claude

Ab Bedrock request ko Anthropic tak route karta hai.

```
Spring AI
  ↓
AWS SDK
  ↓
Bedrock
  ↓
Claude Opus 4.6
```



Claude response generate karta hai.

8. Response Back

Claude:

```
</> JSON
```



```
{
  "content": "Hello! How can I help you?"
}
```



Bedrock



AWS SDK



Spring AI



Java



```
chatClient.prompt(...)
    .call()
    .content();
```



Controller



Browser/Postman

Complete Architecture

Browser/Postman



Spring Boot Controller



ChatClient

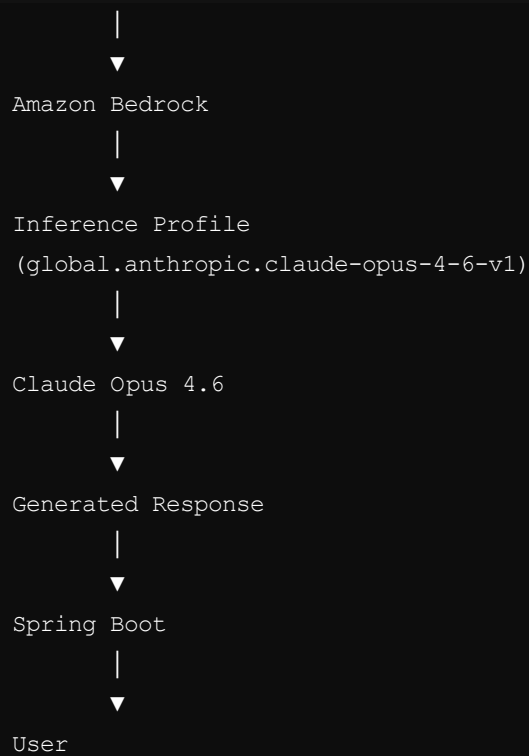


BedrockConverseChatModel



AWS SDK (Bedrock Runtime)





Spring AI ka actual fayda kya hua?

Agar kal tum Bedrock se OpenAI pe switch karna chaho:

Current code:

Java

```
chatClient.prompt(prompt)
    .call()
    .content();
```

Same rahega.

Bas dependency aur yaml change hoga:

YAML

```
spring:
  ai:
    openai:
      api-key: xxxx
```

Ya Gemini:

YAML

```
spring:
  ai:
    vertex:
```

```
ai:  
...
```

Controller code same.

Isi abstraction ko Spring AI provide karta hai.

Agla logical step tumhare liye hoga:

1. ChatClient Advisor
2. PromptTemplate
3. Structured Output
4. Tool Calling
5. RAG with Vector Store
6. Memory (Chat History)

Ye 6 concepts samajh liye to Spring AI ka 80% ecosystem cover ho jayega.